Optional Work Assignment 2 – Part 3
Post-Quantum Secure Handshake Protocol for RTSSP

PQ-SHP/RTSSP/UDP

Alexandre Santos 72970

Miguel Mestre 73018

Contents

1 Introduction

The objective of this optional assignment is to extend the secure real-time media streaming project with a post-quantum handshake protocol. In the previous parts of the project, RTSSP was implemented to protect real-time UDP streaming, and SHP was implemented as a classical secure handshake protocol based on ECDH for key establishment and ECDSA for digital signatures.

This third part introduces **PQ-SHP**, a Post-Quantum Secure Handshake Protocol designed to replace the classical asymmetric cryptography used in SHP. The main objective is to make the handshake resistant to future quantum adversaries while keeping the RTSSP secure streaming layer unchanged.

The final stack for this part is:

Application-Level Real-Time Media Streaming
PQ-SHP – Post-Quantum Secure Handshake Protocol
RTSSP – Real-Time Secure Streaming Protocol
UDP
IP / Data Link

The key design decision is that **only the handshake layer changes**. RTSSP remains responsible for protecting the actual media stream, while PQ-SHP dynamically establishes the session keys that RTSSP will use.

2 Motivation for Post-Quantum SHP

Classical SHP uses:

- **ECDH**: for key agreement.
- **ECDSA**: for digital signatures and authentication.

These algorithms are considered secure against classical computers, but they are not considered secure against sufficiently powerful quantum computers. A quantum attacker using Shor's algorithm could theoretically break elliptic-curve discrete logarithm based schemes such as ECDH and ECDSA.

For this reason, PQ-SHP replaces the classical asymmetric mechanisms with post-quantum alternatives:

Purpose	Classical SHP	PQ-SHP
Key establishment	ECDH	Crystals-Kyber / ML-KEM
Digital signatures	ECDSA	Crystals-Dilithium / ML-DSA
Streaming protection	RTSSP	RTSSP
Symmetric encryption	AES / ChaCha20	AES / ChaCha20

The symmetric protection of RTSSP is preserved because symmetric cryptography is less directly affected by quantum attacks. The main vulnerable part is the asymmetric handshake, which is replaced in this design.

3 System Architecture

3.1 Main Components

The system remains composed of three main components:

- **Streaming Server**: stores the media files and sends RTSSP-protected frames.
- **Secure UDP Proxy**: performs the PQ-SHP handshake, receives the protected stream, decrypts it, and forwards plaintext frames locally.
- **Media Player**: receives the local UDP stream and plays the video. VLC was used for testing.

3.2 Network Flow

The final flow is:

Streaming Server	$\rightarrow$	Secure UDP Proxy	$\rightarrow$	VLC / MPV
RTSSP over UDP		Plain UDP local delivery		Media playback

Before the RTSSP stream starts, the server and proxy execute the PQ-SHP handshake:

Proxy	$\rightarrow$	Server: PQ_SHP_CLIENT_HELLO
Server	$\rightarrow$	Proxy: PQ_SHP_SERVER_HELLO
Proxy	$\rightarrow$	Server: encrypted PQ_SHP_CLIENT_FINAL
Server	$\rightarrow$	Proxy: RTSSP protected stream

4 Cryptographic Design

4.1 Post-Quantum Algorithms

PQ-SHP uses two post-quantum mechanisms:

- **Crystals-Kyber / ML-KEM**: used as a key encapsulation mechanism to establish the RTSSP session secret.
- **Crystals-Dilithium / ML-DSA**: used to sign PQ-SHP handshake messages.

In the implementation, Bouncy Castle's post-quantum provider is used to access Kyber and Dilithium primitives.

4.2 Kyber Key Encapsulation

Kyber is not used exactly like Diffie-Hellman. Instead, it is a Key Encapsulation Mechanism (KEM).

The process is:

1. The proxy generates a Kyber key pair.
2. The proxy sends its Kyber public key in `PQ_SHP_CLIENT_HELLO`.
3. The server encapsulates a shared secret using the proxy's Kyber public key.
4. The server obtains a shared secret and a Kyber encapsulation.
5. The server sends the encapsulation to the proxy in `PQ_SHP_SERVER_HELLO`.
6. The proxy decapsulates the encapsulation using its Kyber private key.
7. Both sides obtain the same shared secret.

4.3 Dilithium Signatures

Dilithium is used to protect the integrity and authenticity of handshake messages.

In PQ-SHP:

- The proxy signs `PQ_SHP_CLIENT_HELLO`.
- The server verifies the proxy signature using the proxy Dilithium public key included in the message.
- The server signs `PQ_SHP_SERVER_HELLO`.
- The proxy verifies the server signature using the server Dilithium public key included in the message.

The signature covers all relevant handshake fields except the signature itself. This ensures that any modification to a signed handshake message is detected during signature verification.

4.4 Runtime Dilithium Keys

In the current implementation, Dilithium key pairs are generated at runtime by both the proxy and the server. Therefore, the protocol verifies message integrity and possession of the corresponding Dilithium private key, but it does not provide persistent identity authentication across different executions.

This means that PQ-SHP authenticates the handshake messages cryptographically, but it does not use a long-term public-key infrastructure or certificates in this version.

5 PQ-SHP Protocol Design

5.1 Protocol Flow

PQ-SHP follows a three-message handshake:

$$\begin{aligned} \text{Proxy} &\rightarrow \text{Server: PQ\_SHP\_CLIENT\_HELLO} \\ \text{Server} &\rightarrow \text{Proxy: PQ\_SHP\_SERVER\_HELLO} \\ \text{Proxy} &\rightarrow \text{Server: encrypted PQ\_SHP\_CLIENT\_FINAL} \end{aligned}$$

After these messages, the server starts the RTSSP protected stream.

5.2 PQ_SHP_CLIENT_HELLO

The proxy sends the first handshake message.

It contains:

- Requested movie name.
- Proxy RTSSP stream endpoint.
- Supported ciphersuites.
- Proxy Kyber public key.
- Client nonce.
- Proxy Dilithium public key.
- Dilithium signature.

Conceptual format:

```
PQ_SHP_CLIENT_HELLO |
movieName |
streamEndpoint |
supportedCipherSuites |
clientKyberPublicKey |
clientNonce |
clientDilithiumPublicKey |
signature
```

The signature is computed over all fields except the signature itself.

5.3 PQ_SHP_SERVER_HELLO

The server replies with:

- Confirmed movie name.
- Selected ciphersuite.

- Server nonce.
- Kyber encapsulation.
- Server Dilithium public key.
- Response to client nonce.
- Dilithium signature.

Conceptual format:

```
PQ_SHP_SERVER_HELLO |
movieName |
selectedCipherSuite |
serverNonce |
kyberEncapsulation |
serverDilithiumPublicKey |
clientNonceResponse |
signature
```

The proxy verifies the server signature before deriving the final RTSSP session keys.

5.4 PQ_SHP_CLIENT_FINAL

After validating `PQ_SHP_SERVER_HELLO`, the proxy derives the same RTSSP session keys and sends an encrypted final message.

Conceptual format:

```
PQ_SHP_CLIENT_FINAL | serverNonceResponse | START_STREAM
```

This message is protected with RTSSP using the session key derived from the Kyber shared secret. If the server successfully decrypts and verifies this message, it starts streaming.

6 Session Key Derivation

The Kyber shared secret is not used directly as an RTSSP key. A derivation step is applied using the shared secret, the two nonces, and a label.

The derivation is:

```
SHA-256(kyberSharedSecret || clientNonce || serverNonce || label)
```

The labels used are:

- `RTSSP PQ ENCRYPTION KEY`
- `RTSSP PQ MAC KEY`

The encryption key is used by RTSSP for AES or ChaCha20. If the negotiated ciphersuite requires a separate MAC, then a separate HMAC key is also derived.

6.1 Conversion to RTSSP Crypto Configuration

The class `PQSHPContext` stores the result of the handshake:

- movie name;
- selected ciphersuite;
- client nonce;
- server nonce;
- Kyber shared secret;
- encryption key;
- MAC key, if needed.

It then creates a `CryptoConfig` object:

```
CryptoConfig crypto = pqContext.toCryptoConfig();
```

This allows the existing RTSSP implementation to be reused without changing the streaming layer.

7 Ciphersuite Negotiation

The proxy advertises its supported RTSSP ciphersuites in `config.properties`:

```
supportedCiphersuites=AES/GCM/NoPadding,ChaCha20-Poly1305,AES/CTR
   /NoPadding
```

The server compares the client's list with its own list:

```
AES/GCM/NoPadding
ChaCha20-Poly1305
AES/CTR/NoPadding
```

The first common ciphersuite is selected. The selected ciphersuite determines how the derived PQ-SHP session key will be used by RTSSP.

8 Implementation Details

8.1 Main Shared Classes

8.1.1 `PQSHP.java`

This class implements the main post-quantum handshake operations:

- Kyber key pair generation.
- Kyber encapsulation.

- Kyber decapsulation.
- Dilithium key pair generation.
- Dilithium signing.
- Dilithium signature verification.
- PQ-SHP message creation.
- PQ-SHP message parsing.
- Nonce generation.
- Nonce challenge/response.
- Ciphersuite negotiation.
- RTSSP key derivation.

8.1.2 `PQSHPContext.java`

This class stores the state resulting from the PQ-SHP handshake and converts it into a `CryptoConfig` object usable by RTSSP.

8.1.3 `RTSSP.java`

RTSSP is reused from the previous parts. It still provides:

- START, DATA, END, and ERROR message types.
- Encryption and decryption.
- Integrity verification.
- Sequence-number based replay protection.

8.2 Server Changes

The server was modified to:

- Receive `PQ_SHP_CLIENT_HELLO`.
- Verify the client Dilithium signature.
- Extract the proxy Kyber public key.
- Encapsulate a Kyber shared secret.
- Generate a server Dilithium key pair.
- Sign `PQ_SHP_SERVER_HELLO`.
- Derive RTSSP session keys from the Kyber shared secret.
- Receive and verify encrypted `PQ_SHP_CLIENT_FINAL`.
- Start RTSSP streaming.

8.3 Proxy Changes

The proxy was modified to:

- Generate a Kyber key pair.
- Generate a Dilithium key pair.
- Send signed `PQ_SHP_CLIENT_HELLO`.
- Receive `PQ_SHP_SERVER_HELLO`.
- Verify the server Dilithium signature.
- Decapsulate the Kyber shared secret.
- Derive RTSSP session keys.
- Send encrypted `PQ_SHP_CLIENT_FINAL`.
- Receive the RTSSP stream and forward plaintext frames to VLC.

9 Expected Runtime Output

9.1 Server Output

Expected server output:

```
Server waiting for signed PQ-SHP CLIENT_HELLO on port 9999
PQ-SHP CLIENT_HELLO received
PQ-SHP CLIENT_HELLO Dilithium signature verified
PQ-SHP SERVER_HELLO sent and signed
Selected CipherSuite: AES/GCM/NoPadding
Kyber shared secret established
RTSSP session keys derived
PQ-SHP CLIENT_FINAL received and verified
PQ-SHP handshake completed
Starting RTSSP stream...
RTSSP START sent
::::::::::::::::::::::::::::
RTSSP END sent
DONE! Frames sent: ...
```

9.2 Proxy Output

Expected proxy output:

```
Proxy listening for RTSSP stream on localhost:8888
Starting PQ-SHP handshake...
PQ-SHP CLIENT_HELLO sent and signed
PQ-SHP SERVER_HELLO received
SERVER_HELLO Dilithium signature verified
Selected CipherSuite: AES/GCM/NoPadding
Kyber shared secret decapsulated
```

```
RTSSP session keys derived
PQ-SHP CLIENT_FINAL sent encrypted
PQ-SHP handshake completed
Waiting for RTSSP stream...

RTSSP START received: START:cars.dat
.........................
RTSSP END received: END:...
Frames received: ...
Packets dropped: 0
```

10 Testing and Validation

10.1 Functional Tests

The following tests were considered for validating the implementation:

Test	Expected Result	Status
Use AES-GCM	Stream decrypts correctly	Passed
Kyber key establishment	Both sides derive the same session key	Passed
Dilithium signatures	Handshake messages are verified	Passed
Encrypted PQ-SHP final message	Server decrypts and verifies final message	Passed
Tamper RTSSP packet	Proxy rejects packet	Implemented
Replay old packet	Proxy rejects packet	Implemented
Modified PQ-SHP message	Signature verification fails	Implemented

10.2 Security Tests

10.2.1 Tampered Handshake Message

If an attacker modifies a signed PQ-SHP message, the Dilithium signature verification fails.

10.2.2 Tampered RTSSP Packet

If an attacker modifies an RTSSP ciphertext, authentication fails and the proxy rejects the packet.

10.2.3 Replay Attack

If an old RTSSP packet is replayed, the proxy rejects it because its sequence number is not greater than the last accepted sequence number.

11 Security Analysis

11.1 Security Properties

PQ-SHP provides the following properties:

- **Post-quantum key establishment**: Kyber replaces ECDH.
- **Post-quantum signatures**: Dilithium replaces ECDSA.
- **Fresh session keys**: nonces and Kyber shared secret are used in key derivation.
- **Handshake integrity**: Dilithium signatures protect the handshake messages.
- **Protected final confirmation**: `PQ_SHP_CLIENT_FINAL` is encrypted using RTSSP and the derived key.
- **Protected streaming**: RTSSP continues to provide confidentiality, integrity, and replay protection.

11.2 Threats Addressed

The implemented design addresses:

- Passive packet sniffing.
- Payload tampering.
- Replay of old RTSSP packets.
- Classical attacks against ECDH and ECDSA by replacing them with post-quantum mechanisms.
- Handshake modification attacks through Dilithium signatures.

11.3 Limitations

The current implementation has the following limitations:

- Dilithium keys are generated at runtime, so the implementation does not provide persistent identity authentication across different executions.
- There is no full post-quantum certificate infrastructure.
- The key derivation function is SHA-256 based and not a full HKDF construction.
- UDP packet loss recovery is not implemented.
- Replay protection uses a simple increasing sequence number rather than a sliding replay window.

- Only one stream session is handled at a time.
- The implementation depends on the correct configuration of the Bouncy Castle PQC provider.

12 Statistics and Experimental Results

Both server and proxy print statistics at the end of the streaming session. The server measures the number of frames sent and the sending throughput, while the proxy measures the number of frames received, dropped packets, and receiving throughput.

12.1 Part 3 – PQ-SHP + RTSSP with Post-Quantum Key Establishment

This test was performed using `cars.dat`. In this version, the server and proxy first executed the PQ-SHP handshake. Kyber was used for post-quantum key establishment, Dilithium was used for post-quantum signatures, and RTSSP used the derived session keys to protect the stream.

12.1.1 Server Statistics

Metric	Observed Value
Frames sent	30515
Movie duration	128 s
Throughput	238 fps
Throughput	2509 Kbps

12.1.2 Proxy Statistics

Metric	Observed Value
Frames received	30515
Packets dropped	0
Throughput	238 fps

The number of frames sent by the server matches the number of frames received by the proxy. The proxy reported zero dropped packets, which indicates that all RTSSP packets were successfully received, authenticated, decrypted, and forwarded to the media player. These results show that replacing the classical SHP handshake with PQ-SHP did not affect the RTSSP streaming behavior in this test.

13 Conclusion

This optional part extends the secure streaming project with a post-quantum handshake protocol. The implemented PQ-SHP protocol replaces the classical ECDH and ECDSA mechanisms used in SHP with Kyber and Dilithium. Kyber is used to establish the RTSSP session secret, while Dilithium is used to sign and verify handshake messages.

The existing RTSSP layer is reused without major changes. This demonstrates a modular protocol design: the handshake mechanism can be replaced while preserving the secure streaming layer. The resulting stack, PQ-SHP/RTSSP/UDP, provides a post-quantum approach to secure real-time UDP streaming.